4-bit Array Multiplier from RTL to GDSII

Digital VLSI Design: SkyWater SKY130 (`sky130_fd_sc_hd`)

Name	Roll number
Gopal	2023112006
Manya	2024102068

Repository: `github.com/ManyaJain111/dvd-assignment`

All steps were run in the IIC-OSIC-TOOLS Docker image [5] (Icarus Verilog 14.0 (devel) [6], Yosys 0.70 [8], OpenSTA 3.1.0 [9], LibreLane [10], KLayout [13]) with the `sky130A` PDK [4], following the assignment handout [1]. The array structure follows the textbook array multiplier [2, 3]. The exact commands, in order, are listed in `README.md`.

1. Part A: RTL design and verification

`rtl/full_adder.v` implements `sum = a ^ b ^ cin` and `cout = ab + a·cin + b·cin` with `assign` statements only. `rtl/mult_array.v` registers `a` and `b` into `a_r`/`b_r` (8 flip-flops), forms the 16 partial products `pp_i[j] = a_r[j] & b_r[i]`, adds them with 12 `full_adder` instances (`fa1_0`…`fa3_3`) wired exactly as in the Section 2 table, and registers the 8-bit result into `p` (8 flip-flops). All 16 flip-flops have an asynchronous active-low reset. The * operator is not used. Figure 1 shows the block diagram.

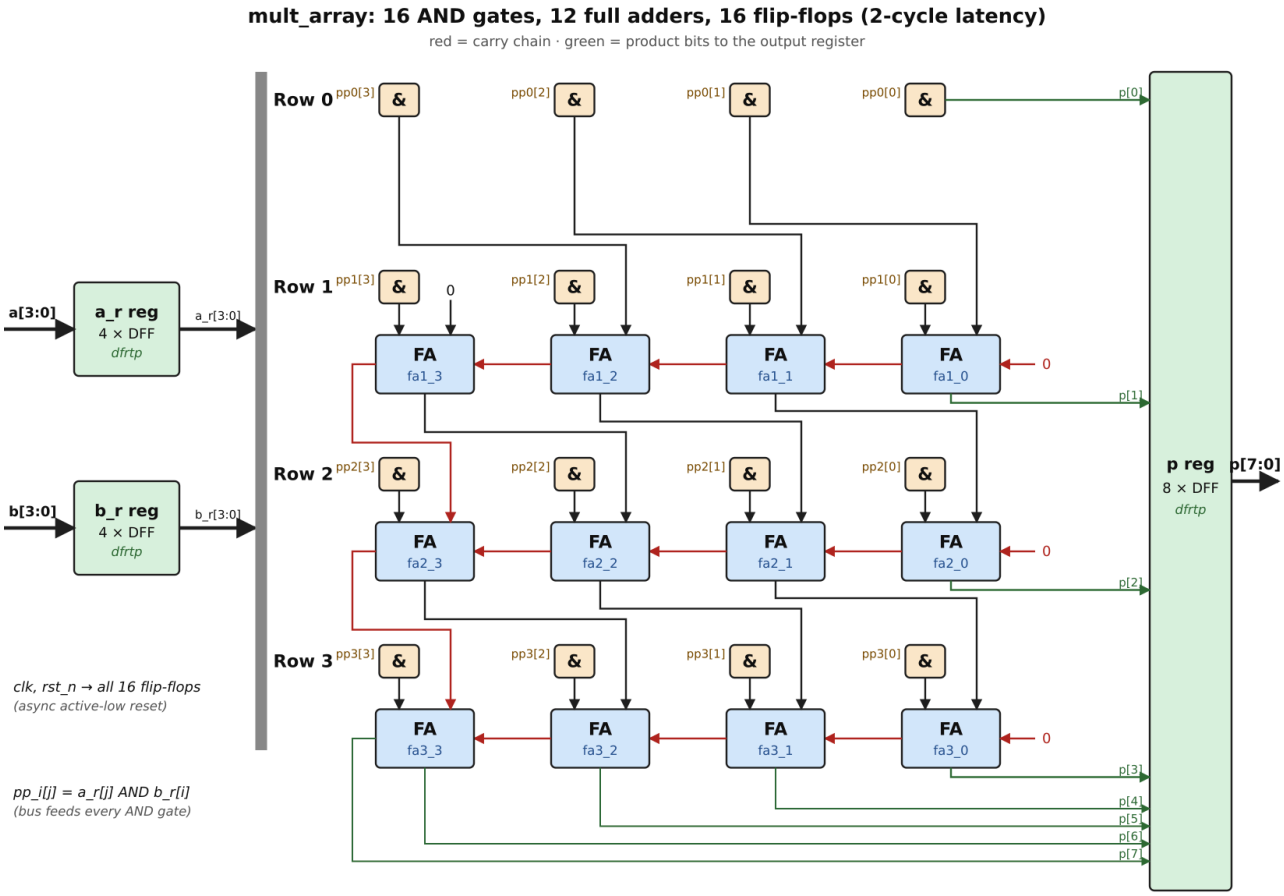


Figure 1: Block diagram: 16 AND gates (orange), 12 full adders (blue), input registers `a_r`, `b_r` and output register `p` (green, 16 flip-flops in total).

The self-checking testbench `tb/tb_mult.v` uses a 10 ns clock, holds `rst_n` low for two cycles, applies all 256 input pairs on the falling edge, waits for two rising edges (the 2-cycle latency), and compares `p` with `a*b` on the next falling edge.

Run	Summary line
RTL simulation	`PASS: 256 vectors checked, 0 errors`
Error test (`sum = a ^ b`)	`FAIL: 256 vectors checked, 95 errors`

The error test drops the carry-in from the sum, so every product that needs a carry inside the array is wrong; 95 of the 256 vectors fail. The change was undone afterwards.

1.1. Hand-worked example

We used Gopal's roll number 2023112006. The last digit is 6, so A = 6 + 5 = 11 and B = 7.

```
          1 0 1 1      A = 11
        x 0 1 1 1      B = 7
-----------------
          1 0 1 1      pp0 = A AND b0 (b0 = 1)
        1 0 1 1        pp1 = A AND b1 (b1 = 1), shifted 1
      1 0 1 1          pp2 = A AND b2 (b2 = 1), shifted 2
    0 0 0 0            pp3 = A AND b3 (b3 = 0), shifted 3
-----------------
0 1 0 0 1 1 0 1      P = 77
```

Through the array rows (Section 2 equations):

- Row 0: p[0] = pp0[0] = 1; r0 = {0, 1, 0, 1}
- Row 1: pp1 + r0 = 1011 + 0101 = 1_0000 → p[1] = 0; r1 = {1, 0, 0, 0}
- Row 2: pp2 + r1 = 1011 + 1000 = 1_0011 → p[2] = 1; r2 = {1, 0, 0, 1}
- Row 3: pp3 + r2 = 0000 + 1001 = 0_1001 → p[3] = 1; p[7:4] = {c3[4], s3[3], s3[2], s3[1]} = 0100
- P = 0100_1101 = 77 = 11 × 7 ✓

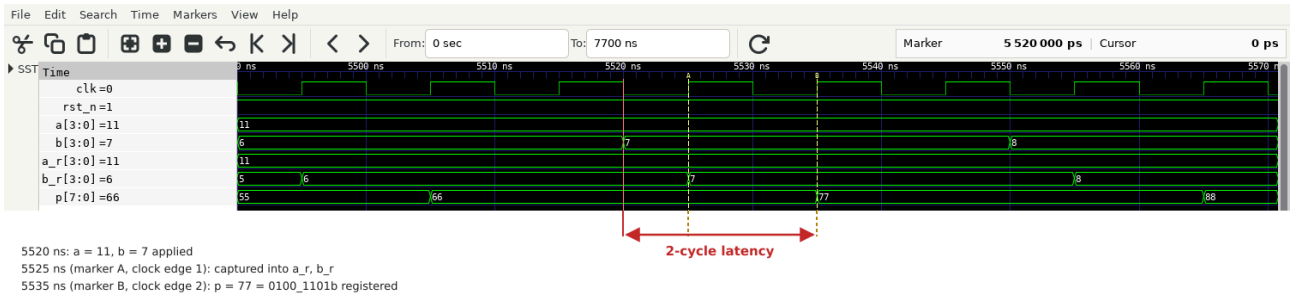


Figure 2: GTKWave view of the RTL simulation for A = 11, B = 7. The inputs change at 5520 ns (red marker), are captured into `a_r`/`b_r` at clock edge 1 (5525 ns, marker A), and the product `p = 77` is registered at clock edge 2 (5535 ns, marker B): a latency of 2 clock edges.

2. Part B: Logic synthesis with Yosys

`synth/synth.tcl` runs `read_verilog`, `synth -top mult_array -flatten`, `dfflibmap`, `abc`, `opt_clean -purge`, `stat` and `write_verilog` with the typical-corner Liberty file.

Metric	Value
Total number of cells	61
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (µm²)	756.976
Flip-flop area (µm²)	16 × 25.024 = 400.384
Flip-flop area (% of total)	52.89 %

3 most-used cell types	`dfrtp_1` × 16, `nand2_1` × 7, `and2_0` × 4 and `maj3_1` × 4 (tied)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

3. Part C: Static timing analysis with OpenSTA

`sta/sta.tcl` uses the typical corner (`tt_025C_1v80`), 1.0 ns input and output delays, and a clock of period T (10 ns by default; overridden with the `STA_PERIOD` environment variable).

Metric	**Value**
Worst slack at T = 10 ns	7.213 ns
T_{min}	$10 - 7.213 = 2.787$ ns
F_{max}	1000 / 2.787 = 358.8 MHz
Slack at T = T_{min} (2.79 ns)	+0.003 ns (≈ 0)
Critical path start point	`_91_/CLK`: the `a_r[1]` flip-flop
Critical path end point	`_88_/D`: the `p[6]` flip-flop
Number of cells on the critical path	9 (launch flip-flop + 8 logic cells)
Latency = 2 × T_{min}	5.574 ns
Throughput	358.8 million results per second

Critical path at T = 10 ns (from `sta/mult_array_timing.rpt`):

#	**Cell**	**Delay (ns)**	**Arrival (ns)**	**Origin**
1	`dfrtp_1` (`a_r[1]`)	0.398	0.398	input register, clock-to-Q
2	`clkinv_1`	0.073	0.471	partial-product logic
3	`o311ai_0`	0.356	0.827	AND gates merged with row 1 adder logic
4	`a21oi_1`	0.153	0.980	AND gates merged with row 1 adder logic
5	`maj3_1`	0.402	1.382	full adder carry
6	`xnor3_1`	0.370	1.752	full adder sum
7	`maj3_1`	0.368	2.119	full adder carry
8	`maj3_1`	0.354	2.474	full adder carry
9	`xor3_1`	0.178	2.652	full adder sum (`fa3_3.sum`) → `p[6]` D pin

Five of the eight logic cells are clean full-adder cells (three `maj3` carry cells and two 3-input XOR/XNOR sum cells). The first three gates come from the partial-product AND gates merged with the first adder row by ABC. This gives 8 logic levels, the same as the ≈ 2N = 8 full-adder delays estimated in Section 2. However, each level is a single library gate rather than a complete two-gate full adder, because ABC uses `maj3`/`xor3` cells that implement a whole carry or sum in one gate.

4. Part D: Physical design with LibreLane

`pnr/config.json` sets `DESIGN_NAME`, `VERILOG_FILES`, `CLOCK_PORT`, `FP_SIZING = "absolute"`, `DIE_AREA = [0, 0, 150, 150]` and `CLOCK_PERIOD`. The starting period was 1.2 × 2.787 = 3.344 ns, rounded up to 3.5 ns. LibreLane signs off setup timing in nine PVT corners; the slow-slow corner (`ss_100C_1v60`) was always the worst one. The period was raised by 0.5 ns after every failed run:

Run	**CLOCK_PERIOD (ns)**	**Worst setup slack (ns)**	**tt-corner slack (ns)**	**DRC / LVS**	**Result**

1	3.5	−2.546	+0.431	0 / 0	setup fail
2	4.0	−2.027	+0.929	0 / 0	setup fail
3	4.5	−1.527	+1.429	0 / 0	setup fail
4	5.0	−1.027	+1.929	0 / 0	setup fail
5	5.5	−0.527	+2.429	0 / 0	setup fail
6	6.0	−0.037	+2.911	0 / 0	setup fail
7	6.5	−0.205	+3.111	0 / 0	setup fail
8	7.0	−0.116	+3.526	0 / 0	setup fail
9	7.5	−0.138	+3.722	0 / 0	setup fail
10	8.0	+0.362	+4.222	0 / 0	**pass**

From run 7 onwards the worst slack stopped improving. With a looser clock, LibreLane's own synthesis and repair steps chose a slower netlist, and the resizer inserted slow `clkdlybuf4s25_1` delay buffers on a long wire to fix max-capacitance. In the slow-slow corner these two buffers alone added about 1.2 ns to the critical path.

Metric	**Value**
CLOCK_PERIOD used (ns)	8.0
Number of runs needed	10
Core area (µm²)	17 759.5 (die area 22 500)
Cells after synthesis (Part B)	61
Cells after place and route	1884 (331 excluding fill and decap cells)
Kinds of cells added	245 `tapvpwrvgnd_1` (tap) · 1299 `decap_3/4/6/8/12` · 254 `fill_1/2` · 17 `buf_2` (I/O buffers) · 3 `clkbuf_16` (clock tree) · 4 `clkdlybuf4s25/50_1` (repair)
Wirelength (µm)	2518 (routed)
Setup slack (ns)	+0.362 (worst corner, `max_ss_100C_1v60`)
Post-route F_{max} (MHz)	1000 / (8.0 − 0.362) = 130.9
F_{max} from Part C (MHz)	358.8
DRC errors	0 (Magic 0, KLayout 0, router 0)
LVS errors	0

The 1884 instances break down as 62 logic and flip-flop cells (LibreLane re-synthesises the RTL itself and got one more cell than Part B), 24 buffers, 245 tap cells and 1553 fill/decap cells: 62 + 24 + 245 + 1553 = 1884.

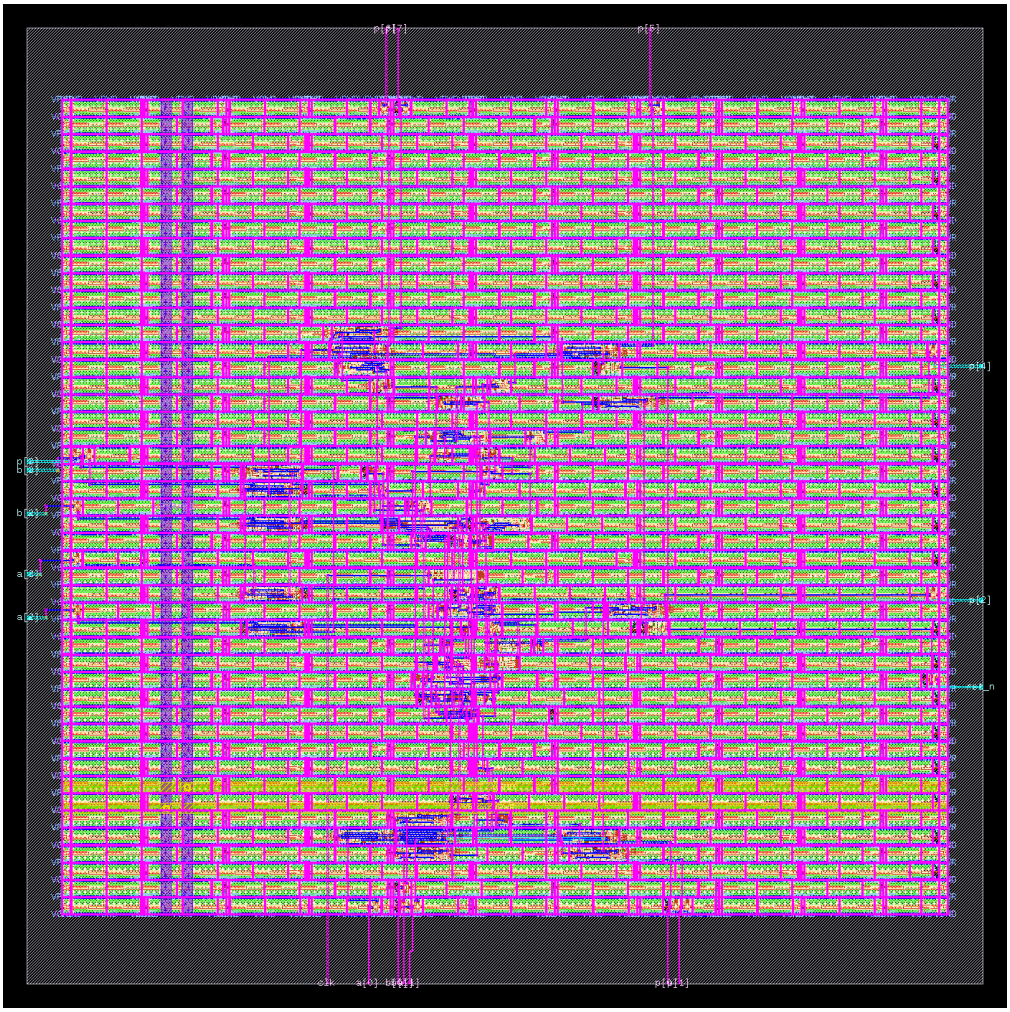


Figure 3: Final layout in KLayout: the whole 150 µm × 150 µm die with the I/O pins, power grid and standard cell rows. The logic sits in the middle; the rest of the rows are filled with decap and fill cells.

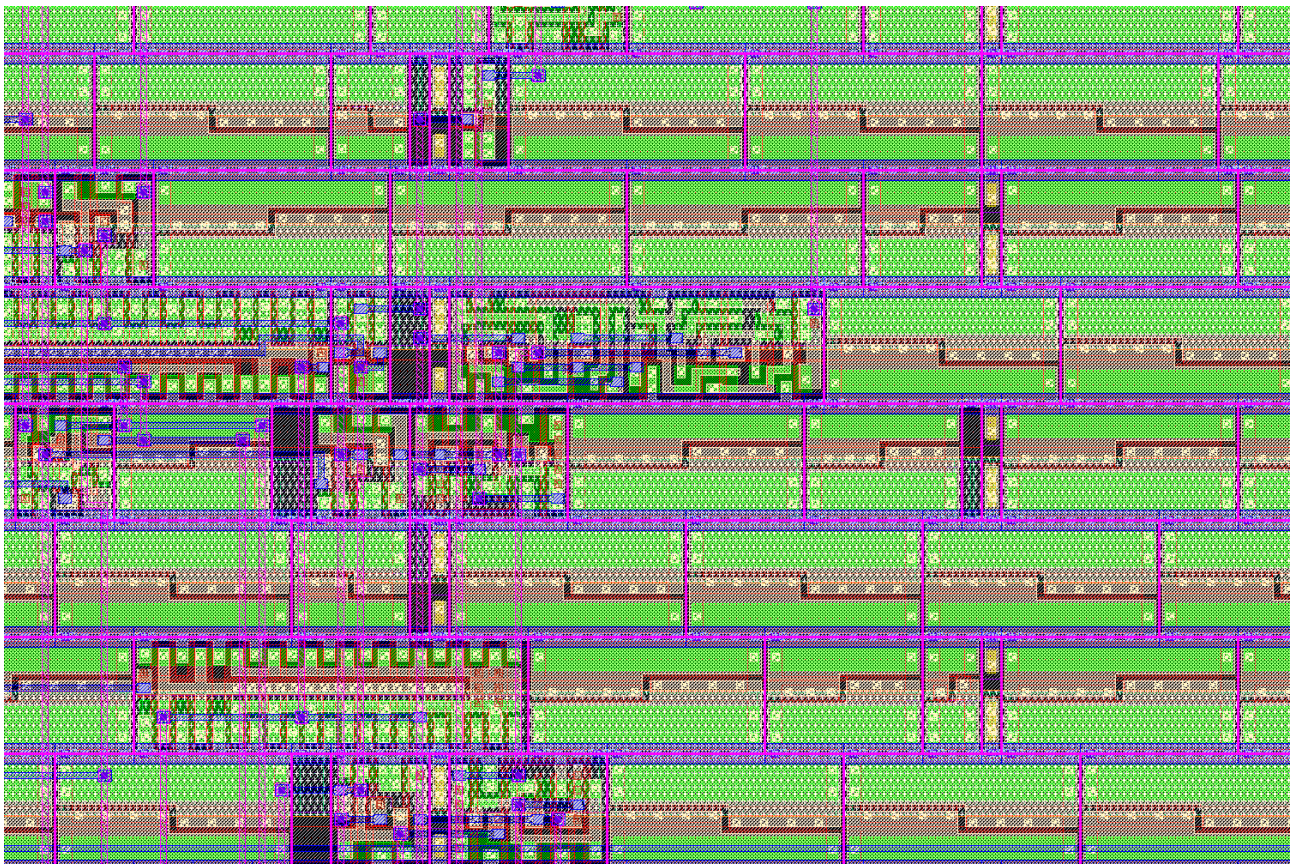


Figure 4: Zoomed view (about 30 µm × 20 µm near the centre of the core): horizontal standard cell rows with their power rails, the placed cells, and metal 1/metal 2 routing with vias.

5. Answers to the questions

1. Cells for the logic. Synthesis produced 61 − 16 = 45 logic cells, not 16 + 12 = 28. A full adder is not one library cell: ABC builds the sum from an `xor3`/`xnor3` (or two `xor2`/`xnor2`) and the carry from a `maj3` or an AOI/OAI combination, so each adder costs about two cells. At the same time, ABC merges and removes logic. Many AND gates are folded into AOI/OAI gates (`o311ai`, `a21oi`, `a22oi`). The adders with a constant 0 input (the `c[0] = 0` carry-ins and `r0[3] = 0`) shrink to half adders or disappear. Some inverters are added for polarity. The result is a gate-level netlist optimised for area, not a one-to-one copy of the AND-gate and full-adder structure.

2. Critical path. The critical path runs from the `a_r[1]` input flip-flop to the `p[6]` output flip-flop. Its data arrival time is 2.652 ns, giving T_{min} = 2.787 ns once the 0.135 ns setup time is added. It contains 9 cells: the launch flip-flop and 8 logic gates. Eight logic levels match the ≈ 2N = 8 full-adder delays estimated for N = 4. The path ripples through all three adder rows (three `maj3` carry cells and two 3-input sum cells come straight from full adders). However, each level is one complex gate of about 0.15–0.40 ns rather than a two-gate full adder.

3. Flip-flop area. The flip-flops take 400.384 of 756.976 µm², i.e. 52.89 % of the cell area. One `dfrtp_1` (25.0 µm²) is a master-slave pair of latches with clock buffering and an asynchronous reset, and is three to six times larger than a simple gate (`nand2_1` 3.75 µm², `maj3_1` 10.0 µm²). The design needs 16 of them for only 45 small logic gates, so they dominate. The number of flip-flops grows as 4N while the logic grows as N², so the flip-flop share falls for wider multipliers.

4. Post-route F_{max}. Post-route F_{max} is 130.9 MHz, compared with 358.8 MHz in Part C. The main reason is the corner. Part C used only the typical corner (25 °C, 1.80 V), but LibreLane signs off the slow-slow corner (100 °C, 1.60 V), where every cell is roughly two to three times slower. In the typical corner the

routed design still reaches 1000 / (8.0 − 4.222) = 264.7 MHz, which is lower than 358.8 MHz for these reasons:

- real wire resistance and capacitance after routing;
- a 0.25 ns clock uncertainty;
- clock-tree latency and skew;
- input/output buffers on every port;
- LibreLane's own area-oriented synthesis and repair buffers (Section 4).

The flow added these cells:

- **Tap cells** (`tapvpwrvgnd`, 245) tie the n-well and substrate to VPWR/VGND at regular intervals to prevent latch-up.
- **Decap cells** (1299) put capacitance between the supply rails to absorb switching current and keep the supply stable.
- **Fill cells** (254) fill the remaining gaps so the wells, implants and power rails are continuous and density rules pass.
- **Buffers** (24): 17 `buf_2` isolate and drive the I/O ports, 3 `clkbuf_16` form the clock tree that delivers the clock to all 16 flip-flops with low skew, and 4 `clkdlybuf` cells repair long-wire and max-capacitance problems.

5. DRC and LVS. DRC (design rule check) checks the layout geometry against the foundry's rules: minimum widths, spacings, enclosures, overlaps and densities on every layer. Only a layout that meets these rules can be printed reliably by the process. LVS (layout versus schematic) extracts the transistors and their connections from the layout and compares them with the netlist, which proves the layout implements exactly the intended circuit (no shorts, opens or missing devices). Both must pass before manufacture: a layout that is DRC-clean but fails LVS gives a working chip of the wrong circuit, and one that passes LVS but fails DRC is the right circuit that cannot be fabricated reliably (it may short, open or fail yield). Our final run has 0 DRC and 0 LVS errors.

6. Scaling to 8 bits. An N-bit array uses N^2 AND gates and $N(N-1)$ full adders. For $N = 8$ this is 64 AND gates and $8 \times 7 = 56$ full adders, against 16 and 12 for $N = 4$: 4.7 × more adders. The critical path is about 2N full-adder delays, i.e. 16 instead of 8, so the logic delay roughly doubles. Using our Part C numbers, the logic part of the path is 2.652 − 0.398 (clock-to-Q) = 2.254 ns for 8 levels (≈ 0.28 ns per level). For $N = 8$: $T_{min} \approx 0.398 + 2 \times 2.254 + 0.135 \approx 5.04$ ns, i.e. $F_{max} \approx 198$ MHz (versus 358.8 MHz). The number of flip-flops grows from 16 to $4N = 32$. The area grows quadratically but the delay only linearly.

Division of work

The work was split by Gopal taking even part numbers and Manya taking odd part numbers. The table below shows the exact contributions. Report parts C and D were written together, while the rest of the report was written by the person who did that part.

Stage	Gopal (2023112006)	Manya (2024102068)
Part A	`rtl/full_adder.v` and `rtl/mult_array.v` (wiring the 12 full adders from the Section 2 table); block diagram	`tb/tb_mult.v` (exhaustive self-checking testbench); RTL simulation; error-injection test; hand-worked example and waveform screenshot
Part B	Reviewed the synthesised netlist against the RTL structure	`synth/synth.tcl`; synthesis run; area table; netlist (gate-level) simulation
Part C	`sta/sta.tcl`; WNS, T_{min} and F_{max} critical-path cell analysis	Re-ran STA at T_{min} to confirm slack ≈ 0; latency and throughput figures
Part D	`pnr/config.json`; the ten LibreLane runs and the timing debugging between them	Extracted metrics from `metrics.json`; pre/post place-and-route cell comparison; KLayout screenshots; DRC/LVS check

Questions	2, 4 and 6	1, 3 and 5
Write-up	Parts C and D of the report	Parts A and B of the report; `README.md` command log

Acknowledgements

We thank the course instructor and teaching assistants for the assignment and its step-by-step handout. We also thank the developers of the open-source tools and the SkyWater SKY130 PDK, and the IIC-OSIC-TOOLS team for packaging them into a single Docker image. Claude, an AI assistant made by Anthropic [15], helped with writing scripts, debugging the flow, and editing this report and the README. All results were produced by running the tools ourselves and were checked by the team.

References

[1] Course handout, *RTL to GDSII: 4-bit Array Multiplier on SKY130* (`RTL_to_GDS.pdf`), Digital VLSI Design, 2026.

[2] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Addison-Wesley, 2011, ch. 11 (datapath subsystems: multiplication).

[3] J. M. Rabaey, A. Chandrakasan and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Prentice Hall, 2003, ch. 11 (arithmetic building blocks).

[4] SkyWater Technology and Google, *SKY130 open-source PDK*. https://github.com/google/skywater-pdk

[5] Institute for Integrated Circuits, JKU Linz, *IIC-OSIC-TOOLS*. https://github.com/iic-jku/IIC-OSIC-TOOLS

[6] S. Williams, *Icarus Verilog*. https://github.com/steveicarus/iverilog

[7] *GTKWave* waveform viewer. https://github.com/gtkwave/gtkwave

[8] C. Wolf et al., *Yosys Open SYnthesis Suite*. https://github.com/YosysHQ/yosys

[9] J. Cherry, *OpenSTA static timing analyzer*. https://github.com/parallaxsw/OpenSTA

[10] *LibreLane* RTL-to-GDSII flow. https://github.com/librelane/librelane

[11] T. Edwards, *Magic VLSI layout tool*. http://opencircuitdesign.com/magic

[12] T. Edwards, *Netgen LVS tool*. http://opencircuitdesign.com/netgen

[13] M. Köfferlein, *KLayout*. https://www.klayout.de

[14] *Typst* typesetting system (used to write this report). https://typst.app

[15] Anthropic, *Claude* (Claude Opus 5.5 via Claude Code), AI assistant, used October 2026. https://www.anthropic.com/claude